

Σημειώσεις Θεωρίας Αναγνώριση Προτύπων

Μιχαήλ-Αθανάσιος Πέππας

26 Νοεμβρίου 2025

Περιεχόμενα

1	Μέρος 1: Ταξινόμηση στη Μία Διάσταση	3
1.1	Εισαγωγή: πρόβλημα, οικογένειες μοντέλων και βασικές έννοιες	3
1.2	Μη παραμετρικοί ταξινομητές: k -Nearest Neighbors	4
1.3	Γραμμικοί διακριτικοί ταξινομητές: Perceptron και LMS	5
1.4	Γενετικά Γκαουσιανά μοντέλα και Ταξινομητής Bayes	6
1.5	Συγκεντρωτική σύγκριση μεθόδων (Μέρος 1)	8
2	Μέρος 2: Κρυφά Μοντέλα Markov (HMM)	9
2.1	Θεωρητικό πλαίσιο: στοχαστικές ακολουθίες και ιδιότητα Markov	9
2.2	Ορισμός HMM ως γενετικού μοντέλου	9
2.3	Τα τρία βασικά προβλήματα σε ένα HMM	10
2.4	Πρόβλημα αξιολόγησης: forward αλγόριθμος	10
2.5	Πρόβλημα αποκωδικοποίησης: αλγόριθμος Viterbi	11
2.6	Πρόβλημα εκμάθησης: EM / Baum–Welch	11
2.7	Viterbi-training (hard EM) και σύγκριση	12
2.8	Σχέση HMM με τα μοντέλα του Μέρους 1	13
3	Μέρος 3: Backpropagation σε Απλό Νευρωνικό Δίκτυο	14
3.1	Νευρωνικά δίκτυα ως διακριτικά, μη γραμμικά μοντέλα	14
3.2	Υπολογιστικός γράφος και κανόνας αλυσίδας	14
3.3	Τυπικό 2-στρώσης δίκτυο και συνάρτηση κόστους	15
3.4	Αναλυτικός υπολογισμός $\partial L / \partial w_1$	15
3.5	Διατύπωση με σήματα σφάλματος δ (error signals)	16
3.6	Συγκρίσεις με LMS, Perceptron και HMM	16
3.7	Σημεία-κλειδιά για θεωρητικές ερωτήσεις backprop	16
4	Μέρος 4: Expectation–Maximization (EM)	17
4.1	Θεωρητικό πλαίσιο: ML με κρυφές μεταβλητές	17
4.2	Το συγκεκριμένο παράδειγμα: εκθετική $\times$ ομοιόμορφη	18
4.3	(α) E-step: κατασκευή του $Q(\theta, \theta^{(0)})$	18
4.4	(β) M-step: ML εκτίμηση παραμέτρων	20
4.5	(γ) Γραφική ερμηνεία και σύνδεση με άλλα μοντέλα	20
5	Μέρος 5: Bayes Network – Μοντέλο Chest Clinic	22
5.1	Θεωρία Bayesian Networks	22
5.2	Το μοντέλο Chest Clinic ως παράδειγμα BN	22
5.3	(α) Παραγοντοποίηση της joint κατανομής	23
5.4	(β) Συνθηκές ανεξαρτησίας και d-separation	23

5.5	(γ) Υπολογισμός πιθανοτήτων με product-sum	24
5.6	Moralization, triangulation και αποδοτικό inference	24
5.7	Σύνδεση με τα υπόλοιπα μέρη	25
6	Μέρος 6: Απλό RNN που Μετράει Μονάδες	26
6.1	RNN ως δυναμικό διακριτικό μοντέλο	26
6.2	Ειδικό μοντέλο: γραμμικό RNN που μετράει μονάδες	26
6.3	(α) Backpropagation Through Time (BPTT) και gradients	27
6.4	(β) Παράδειγμα ενός update με αρχικά βάρη	28
6.5	(γ) Καλύτερη αρχικοποίηση και ιδανική λύση	28
6.6	(δ) Vanishing / exploding gradients και συγκρίσεις	29

1 Μέρος 1: Ταξινόμηση στη Μία Διάσταση

1.1 Εισαγωγή: πρόβλημα, οικογένειες μοντέλων και βασικές έννοιες

Μελετάμε ένα κλασικό πρόβλημα δυαδικής ταξινόμησης με ένα μόνο χαρακτηριστικό:

$$x \in \mathbb{R}, \quad y \in \{A, B\}.$$

Το σενάριο αυτό είναι σκόπιμα απλό, αλλά θεωρητικά πολύ πλούσιο, γιατί επιτρέπει να συγκρίνουμε διαφορετικές οικογένειες μοντέλων ταξινόμησης:

- Μη παραμετρικές μέθοδοι, όπως ο k -Nearest Neighbors (k -NN).
- Γραμμικοί διακριτικοί ταξινομητές (Perceptron, LMS).
- Γενετικά μοντέλα με Γκαουσιανές ανά κλάση και ταξινομητή Bayes.

Στόχος μας είναι θεωρητικός: να καταλάβουμε

- πώς διαφέρουν αυτές οι οικογένειες,
- τι υποθέσεις κάνουν για τα δεδομένα,
- πότε προσεγγίζουν τον βέλτιστο ταξινομητή Bayes.

Βασικές έννοιες:

Discriminative vs generative • **Generative** μοντέλα: μοντελοποιούν ολόκληρη την κατανομή

$$p(x, y) = P(C) p(x | C),$$

π.χ. Γκαουσιανές ανά κλάση, HMM. Από αυτά προκύπτει ο ταξινομητής μέσω του κανόνα Bayes.

- **Discriminative** μοντέλα: μοντελοποιούν άμεσα την απόφαση ή την $p(y | x)$, χωρίς να ορίζουν ρητά $p(x | y)$. Παραδείγματα: Perceptron, LMS, Logistic Regression, Νευρωνικά Δίκτυα.

Parametric vs non-parametric • **Parametric**: η “γνώση” συμπυκνώνεται σε έναν μικρό αριθμό παραμέτρων (π.χ. μέσοι, διασπορές, βάρη).

- **Non-parametric**: δεν υπάρχει προκαθορισμένος περιορισμένος αριθμός παραμέτρων· το μοντέλο “μεγαλώνει” με τα δεδομένα (π.χ. k -NN).

Γραμμικά vs μη γραμμικά όρια απόφασης • Στη 1-D, γραμμικό όριο σημαίνει ένα μόνο σημείο κατωφλίου x^* .

- Μη γραμμικό όριο μπορεί να αποτελείται από πολλά διαστήματα (π.χ. $A, B, A, B, \dots$), όπως στον 1-NN.

Bayes βέλτιστος ταξινομητής • Θεωρητικά, ο καλύτερος ταξινομητής είναι αυτός που ελαχιστοποιεί την πιθανότητα λάθους:

$$y^*(x) = \arg \max_C P(C | x).$$

- Το αντίστοιχο ελάχιστο σφάλμα ονομάζεται **Bayes σφάλμα**.

Η ταξινόμηση στη μία διάσταση είναι έτσι μια “μικρογραφία” όλων αυτών των ιδεών: κάθε μέθοδος δίνει διαφορετικό σχήμα για το όριο απόφασης και κάνει διαφορετικές υποθέσεις για το πώς προκύπτουν τα δεδομένα.

1.2 Μη παραμετρικοί ταξινομητές: k -Nearest Neighbors

Διαισθητική ιδέα και χαρακτηριστικά

Ο ταξινομητής k -Nearest Neighbors (k -NN):

- Δεν μαθαίνει ρητές παραμέτρους (όπως βάρη ή μέσους).
- Αποθηκεύει όλα τα εκπαιδευτικά ζεύγη $\{(x_n, y_n)\}_{n=1}^N$.
- Για ένα νέο x , βρίσκει τους k κοντινότερους γείτονες (στη 1-D: με βάση την απόσταση $|x - x_n|$).
- **Απόφαση:** επιλέγει την κλάση που εμφανίζεται πιο συχνά μεταξύ αυτών των k δειγμάτων.

Άρα:

- Είναι **μη παραμετρικός**: δεν υποθέτει συγκεκριμένη μορφή για $p(x | C)$.
- Είναι **βασισμένος σε παραδείγματα** (instance-based): η γνώση μένει στα ίδια τα παραδείγματα, όχι σε ένα μικρό διάνυσμα παραμέτρων.

Όρια απόφασης στη μία διάσταση

Στη 1-D τα δεδομένα τοποθετούνται πάνω στον άξονα των πραγματικών.

- Για 1-NN:
 - Κάθε σημείο x ταξινομείται στην κλάση του *πλησιέστερου* δείγματος.
 - Τα **όρια απόφασης** είναι τα *midpoints* μεταξύ διαδοχικών δειγμάτων διαφορετικής κλάσης.
 - Το όριο απόφασης είναι πολύ “οδοντωτό” και προσαρμόζεται ακριβώς σε κάθε σημείο του training set.
- Για $k > 1$ (π.χ. 3-NN):
 - Η απόφαση είναι με **πλειοψηφία**.
 - Τα όρια απόφασης γίνονται πιο “ομαλά” (λιγότερες εναλλαγές κλάσης).

Από την οπτική bias–variance:

- **Μικρό k :** χαμηλό bias, υψηλό variance (έντονη προσαρμογή και σε θόρυβο).
- **Μεγάλο k :** υψηλότερο bias, χαμηλότερο variance (πιο “γενική” εικόνα).

Θεωρητική συμπεριφορά και συγκρίσεις

- Θεωρητικά, για $N \rightarrow \infty$ και κατάλληλη επιλογή k (που αυξάνει με N), ο k -NN μπορεί να προσεγγίσει τον Bayes ταξινομητή.
- Υπολογιστικά, είναι **ακριβός στο test**: για κάθε νέο x χρειάζεται να βρει τις αποστάσεις από όλα τα εκπαιδευτικά δείγματα.
- Σε αντίθεση με τα γραμμικά μοντέλα (Perceptron/LMS), **μπορεί να παράγει μη γραμμικά όρια απόφασης** ακόμη και στη 1-D (πολλά διαστήματα).
- Δεν δίνει άμεσα θεωρητική ερμηνεία μέσω παραμέτρων (π.χ. δεν μαθαίνει μέσους/διασπορές).

1.3 Γραμμικοί διακριτικοί ταξινομητές: Perceptron και LMS

Και τα δύο μοντέλα ανήκουν στην ίδια οικογένεια: **γραμμικοί διακριτικοί ταξινομητές**. Στη μία διάσταση, το μοντέλο έχει τη μορφή

$$y(x) = wx + b.$$

Στο τέλος, η απόφαση προκύπτει συγκρίνοντας το $y(x)$ με κάποιο κατώφλι.

Perceptron

Μοντέλο και όριο απόφασης. Στην απλούστερη μορφή:

$$\hat{y}(x) = \text{sign}(wx + b),$$

όπου $\hat{y} \in \{-1, +1\}$ κωδικοποιεί τις δύο κλάσεις (A, B) . Το **όριο απόφασης** είναι

$$wx + b = 0 \Rightarrow x^* = -\frac{b}{w},$$

ένα μοναδικό σημείο διαχωρισμού στη 1-D (γραμμικό όριο).

Κανόνας εκμάθησης (Perceptron rule). Για κάθε δείγμα (x_n, t_n) με $t_n \in \{-1, +1\}$:

- Υπολογίζουμε $\hat{y}_n = \text{sign}(wx_n + b)$.
- Αν $\hat{y}_n = t_n$, δεν κάνουμε τίποτα.
- Αν $\hat{y}_n \neq t_n$, ενημερώνουμε:

$$w \leftarrow w + \eta t_n x_n, \quad b \leftarrow b + \eta t_n,$$

όπου $\eta > 0$ είναι ο ρυθμός μάθησης.

Μια **epoch** = ένα πλήρες πέρασμα από όλα τα δείγματα.

Θεωρία και ιδιότητες.

- **Perceptron convergence theorem:** αν τα δεδομένα είναι **γραμμικά διαχωρίσιμα**, το perceptron συγκλίνει σε πεπερασμένα βήματα σε κάποιο (w, b) που διαχωρίζει τέλεια τις δύο κλάσεις.
- Αν τα δεδομένα **δεν** είναι διαχωρίσιμα (π.χ. αλληλοεπικαλυπτόμενες Γκαουσιανές), το perceptron δεν συγκλίνει· συνεχίζει να τροποποιεί τα βάρη.
- Είναι **διακριτικό** και **παραμετρικό** μοντέλο: μαθαίνει απευθείας ένα γραμμικό όριο, χωρίς να μοντελοποιεί $p(x | C)$.

LMS (Widrow–Hoff): γραμμική παλινδρόμηση για ταξινόμηση

Ο LMS αντιμετωπίζει την ταξινόμηση ως **γραμμική παλινδρόμηση**:

$$y(x) = wx + b,$$

και επιλέγει τιμές στόχου t_n (π.χ. $t_n \in \{-1, +1\}$) έτσι ώστε $y(x_n) \approx t_n$. Το όριο απόφασης προκύπτει πάλι από το σημείο όπου $y(x) = 0$ ή από ένα κατάλληλο κατώφλι.

Συνάρτηση κόστους. Για εκπαίδευση χρησιμοποιείται η μέση τετραγωνική απόκλιση (MSE):

$$J(w, b) = \frac{1}{2} \sum_{n=1}^N (t_n - (wx_n + b))^2.$$

Gradient descent και κανόνας ενημέρωσης. Παίρνοντας παραγώγους:

$$\frac{\partial J}{\partial w} = - \sum_{n=1}^N (t_n - y_n) x_n, \quad \frac{\partial J}{\partial b} = - \sum_{n=1}^N (t_n - y_n),$$

όπου $y_n = wx_n + b$.

Με **online** gradient descent (ένα δείγμα τη φορά):

$$w \leftarrow w + \eta(t_n - y_n)x_n, \quad b \leftarrow b + \eta(t_n - y_n).$$

Συγκρίσεις Perceptron vs LMS.

- Και τα δύο δίνουν **γραμμικό όριο απόφασης** στη 1-D.
- Perceptron: ενημερώνει μόνο αν υπάρχει *λάθος ταξινόμηση*, δεν ενδιαφέρεται για το “πόσο” λάθος είναι.
- LMS: ελαχιστοποιεί το **μέγεθος του σφάλματος** (MSE), άρα είναι πιο “ομαλό” και σταθερό σε θορυβώδη δεδομένα.
- Και τα δύο είναι **διακριτικά, παραμετρικά** μοντέλα και δουλεύουν καλά όταν το πραγματικό όριο απόφασης είναι περίπου γραμμικό.

1.4 Γενετικά Γκαουσιανά μοντέλα και Ταξινομητής Bayes

Γενετικό μοντέλο με Γκαουσιανές ανά κλάση

Υποθέτουμε ότι για κάθε κλάση $C \in \{A, B\}$ η κατανομή του x είναι Γκαουσιανή:

$$p(x | C) = \mathcal{N}(x | \mu_C, \sigma_C^2) = \frac{1}{\sqrt{2\pi\sigma_C^2}} \exp\left(-\frac{(x - \mu_C)^2}{2\sigma_C^2}\right).$$

Επιπλέον, έχουμε priors $P(A), P(B)$ με $P(A) + P(B) = 1$. Το μοντέλο είναι **γενετικό**: περιγράφει πώς “παράγονται” τα δεδομένα.

ML εκτίμηση για μέσο και διασπορά

Για τα δείγματα της κλάσης C , $\{x_1, \dots, x_{N_C}\}$, η **μέγιστη πιθανοφάνεια** (ML) δίνει:

$$\mu_C^{\text{ML}} = \frac{1}{N_C} \sum_{n=1}^{N_C} x_n,$$
$$\sigma_C^{2\text{ML}} = \frac{1}{N_C} \sum_{n=1}^{N_C} (x_n - \mu_C^{\text{ML}})^2.$$

Πρόκειται για τις αναμενόμενες εκφράσεις: ο δειγματικός μέσος και η δειγματική διασπορά με παρονομαστή N_C (ML, όχι unbiased).

Ταξινομητής Bayes και όριο απόφασης

Ο ταξινομητής Bayes αποφασίζει υπέρ της κλάσης με τη μεγαλύτερη posterior:

$$\text{απόφαση}(x) = \begin{cases} A, & \text{αν } P(A | x) > P(B | x), \\ B, & \text{αλλιώς.} \end{cases}$$

Με τον κανόνα Bayes:

$$P(A | x) = \frac{P(A)p(x | A)}{p(x)}, \quad P(B | x) = \frac{P(B)p(x | B)}{p(x)},$$

όπου $p(x) = P(A)p(x | A) + P(B)p(x | B)$.

Το όριο απόφασης x^* ικανοποιεί:

$$P(A)p(x^* | A) = P(B)p(x^* | B).$$

Αν οι δύο Γκαουσιανές έχουν ίση διασπορά $\sigma_A^2 = \sigma_B^2 = \sigma^2$, τότε μετά από λογαρίθμηση παίρνουμε γραμμικό όρο στο x και άρα ένα μοναδικό σημείο:

$$x^* = \frac{2\sigma^2 \ln\left(\frac{P(B)}{P(A)}\right) - (\mu_A^2 - \mu_B^2)}{2(\mu_B - \mu_A)}.$$

Ειδική περίπτωση $P(A) = P(B)$:

$$x^* = \frac{\mu_A + \mu_B}{2}.$$

Bayes σφάλμα και απόσταση των Γκαουσιανών

Ο Bayes ταξινομητής ελαχιστοποιεί την πιθανότητα σφάλματος

$$P_e = \int_{\mathcal{R}_A} P(B)p(x | B) dx + \int_{\mathcal{R}_B} P(A)p(x | A) dx,$$

ή ισοδύναμα

$$P_e = \int_{-\infty}^{+\infty} \min\{P(A)p(x | A), P(B)p(x | B)\} dx.$$

Για δύο Γκαουσιανές με ίσες διασπορές σ^2 και ίσους priors, αν ορίσουμε

$$d = \frac{|\mu_B - \mu_A|}{\sigma},$$

τότε (χωρίς απόδειξη):

$$P_e = \Phi\left(-\frac{d}{2}\right),$$

όπου Φ η CDF της $\mathcal{N}(0, 1)$. Όσο μεγαλύτερο d , τόσο πιο καλά διαχωρίζονται οι κλάσεις και τόσο μικρότερο το Bayes σφάλμα.

Συγκρίσεις με διακριτικούς ταξινομητές

- Τα γενετικά Γκαουσιανά μοντέλα **μοντελοποιούν το** $p(x \mid C)$, κάτι που επιτρέπει όχι μόνο ταξινόμηση αλλά και “μοντελοποίηση των δεδομένων”.
- Αν οι πραγματικές κατανομές είναι κοντά σε Γκαουσιανές, το μοντέλο μπορεί να πλησιάσει πολύ τον βέλτιστο Bayes ταξινομητή.
- Σε αντίθεση, Perceptron και LMS δεν έχουν έννοια “κατανομής”, αλλά μαθαίνουν *άμεσα* το όριο απόφασης.
- Στη 1-D, τόσο το Γκαουσιανό μοντέλο (με ίσες διασπορές) όσο και ο γραμμικός ταξινομητής παράγουν ουσιαστικά ένα *μοναδικό κατώφλι*.

1.5 Συγκεντρωτική σύγκριση μεθόδων (Μέρος 1)

Μέθοδος	Generative	Parametric	Όριο στη 1-D	Ευελιξία
k -NN	όχι	όχι	πολλαπλά διαστήματα	πολύ υψηλή (τοπικό)
Perceptron	όχι	ναι	ένα κατώφλι	μέτρια, γραμμικό
LMS	όχι	ναι	ένα κατώφλι	μέτρια, γραμμικό
Γκαουσιανό + Bayes	ναι	ναι	συνήθως ένα κατώφλι	υψηλή αν υπόθεση σωστή

Σημεία-κλειδιά:

- k -NN: δυνατός μη παραμετρικός ταξινομητής, αλλά ακριβός στο test και δύσκολος σε μεγάλες διαστάσεις.
- Perceptron/LMS: απλά διακριτικά, παραμετρικά μοντέλα· χρήσιμα για κατανόηση γραμμικών ορίων και gradient descent.
- Γκαουσιανό generative μοντέλο: συνδέει ταξινόμηση με πιθανοτικές υποθέσεις και Bayes βελτιστοτητα.

2 Μέρος 2: Κρυφά Μοντέλα Markov (HMM)

2.1 Θεωρητικό πλαίσιο: στοχαστικές ακολουθίες και ιδιότητα Markov

Σε πολλά προβλήματα (ομιλία, βιολογικές ακολουθίες, NLP) η **τάξη** των παρατηρήσεων παίζει κεντρικό ρόλο:

$$O_0, O_1, \dots, O_T.$$

Είναι ανεπαρκές να αντιμετωπίζουμε τα O_t ως ανεξάρτητα· χρειάζεται μοντέλο που να περιγράφει πώς εξελίσσεται η κατάσταση στο χρόνο.

Η **ιδιότητα Markov πρώτης τάξης** λέει ότι:

$$P(q_t \mid q_{0:t-1}) = P(q_t \mid q_{t-1}),$$

δηλαδή η επόμενη κατάσταση εξαρτάται μόνο από την *τρέχουσα*.

Τα HMM συνδυάζουν:

- αλυσιδωτή δομή Markov για τις κρυφές καταστάσεις,
- πιθανότητες εκπομπής για τις παρατηρήσεις.

2.2 Ορισμός HMM ως γενετικού μοντέλου

Ένα HMM ορίζεται από τις παραμέτρους $\lambda = (\pi, A, B)$:

- Καταστάσεις $q_t \in \{1, \dots, S\}$.
- Παρατηρήσεις O_t (διακριτές ή συνεχείς).
- Αρχική κατανομή $\pi_i = P(q_0 = i)$.
- Μεταβατικές πιθανότητες $a_{ij} = P(q_t = j \mid q_{t-1} = i)$.
- Πιθανότητες εκπομπής $b_j(o) = P(O_t = o \mid q_t = j)$.

Γενετική διαδικασία.

1. Επιλέγουμε αρχική κατάσταση q_0 από την π .
2. Για $t = 0, 1, \dots, T$:
 - εκπέμπουμε παρατήρηση O_t από την κατανομή $b_{q_t}(\cdot)$,
 - μεταβαίνουμε σε νέα κατάσταση q_{t+1} με πιθανότητα $a_{q_t, q_{t+1}}$.

Η **joint** πιθανότητα διαδρομής και παρατηρήσεων είναι:

$$P(q_0, \dots, q_T, O_0, \dots, O_T) = \pi_{q_0} b_{q_0}(O_0) \prod_{t=1}^T a_{q_{t-1}, q_t} b_{q_t}(O_t).$$

Όπως τα Γκαουσιανά μοντέλα στο Μέρος 1, έτσι κι εδώ έχουμε ένα **γενετικό μοντέλο**, μόνο που τώρα το x αντικαθίσταται από ολόκληρη την ακολουθία $O_{0:T}$.

2.3 Τα τρία βασικά προβλήματα σε ένα HMM

Για ένα HMM και μια ακολουθία παρατηρήσεων $O_{0:T}$, τα κλασικά θεωρητικά προβλήματα είναι:

1. Πρόβλημα αξιολόγησης:

να υπολογίσουμε $P(O_{0:T} \mid \lambda)$.

2. Πρόβλημα αποκωδικοποίησης:

να βρούμε την πιο πιθανή ακολουθία καταστάσεων $q_{0:T}^*$.

3. Πρόβλημα εκμάθησης:

να εκτιμήσουμε τις παραμέτρους λ από παρατηρούμενες ακολουθίες.

Τα αντίστοιχα βασικά εργαλεία είναι:

- **Forward** (ή forward-backward) αλγόριθμος,
- **Viterbi** αλγόριθμος,
- **EM / Baum-Welch** (και παραλλαγές όπως Viterbi-training).

2.4 Πρόβλημα αξιολόγησης: forward αλγόριθμος

Η “άμεση” λύση του $P(O_{0:T})$ είναι:

$$P(O_{0:T}) = \sum_{q_{0:T}} P(q_{0:T}, O_{0:T}),$$

άθροισμα σε όλες τις δυνατές διαδρομές καταστάσεων· αυτό όμως είναι εκθετικό σε T .

Ο **forward αλγόριθμος** χρησιμοποιεί **δυναμικό προγραμματισμό**.

Ορίζουμε:

$$\alpha_t(j) = P(O_0, \dots, O_t, q_t = j).$$

Αρχικοποίηση.

$$\alpha_0(j) = \pi_j b_j(O_0), \quad j = 1, \dots, S.$$

Επανάληψη. Για $t = 1, \dots, T$:

$$\alpha_t(j) = b_j(O_t) \sum_{i=1}^S \alpha_{t-1}(i) a_{ij}.$$

Τερματισμός.

$$P(O_{0:T}) = \sum_{j=1}^S \alpha_T(j).$$

Ερμηνεία:

- Η $\alpha_t(j)$ είναι η πιθανότητα να έχουμε παρατηρήσει $O_0, \dots, O_t$ και να βρισκόμαστε στην κατάσταση j στο t .
- Ο αλγόριθμος “συγκεντρώνει” όλες τις διαδρομές που καταλήγουν σε κάθε j χωρίς να τις απαριθμεί μία-μία.

2.5 Πρόβλημα αποκωδικοποίησης: αλγόριθμος Viterbi

Τώρα θέλουμε την **πιο πιθανή διαδρομή καταστάσεων**:

$$q_0^*, \dots, q_T^* = \arg \max_{q_{0:T}} P(q_{0:T}, O_{0:T}).$$

Ο **Viterbi** αλγόριθμος μοιάζει με τον forward, αλλά αντί για άθροισμα κρατάει το **μέγιστο**. Ορίζουμε:

$$\delta_t(j) = \max_{q_{0:t-1}} P(q_{0:t-1}, q_t = j, O_{0:t}),$$

και δείκτες backtracking

$$\psi_t(j) = \arg \max_i [\delta_{t-1}(i) a_{ij}].$$

Αρχικοποίηση.

$$\delta_0(j) = \pi_j b_j(O_0), \quad \psi_0(j) = 0.$$

Επανάληψη.

$$\delta_t(j) = b_j(O_t) \max_i [\delta_{t-1}(i) a_{ij}], \quad \psi_t(j) = \arg \max_i [\delta_{t-1}(i) a_{ij}].$$

Τερματισμός & backtracking.

$$P^* = \max_j \delta_T(j), \quad q_T^* = \arg \max_j \delta_T(j),$$

και για $t = T - 1, \dots, 0$:

$$q_t^* = \psi_{t+1}(q_{t+1}^*).$$

Σημαντική διαφορά:

- Forward: υπολογίζει **ολική πιθανότητα** $P(O_{0:T})$ (άθροισμα όλων των διαδρομών).
- Viterbi: υπολογίζει **μια διαδρομή μέγιστης πιθανότητας** (MAP path).

2.6 Πρόβλημα εκμάθησης: EM / Baum–Welch

Θέλουμε να εκτιμήσουμε $\lambda = (\pi, A, B)$ από παρατηρήσεις $\mathcal{D} = \{O_{0:T_m}^{(m)}\}_m$.

Οι καταστάσεις q_t είναι κρυφές, άρα η άμεση ML βελτιστοποίηση είναι δύσκολη. Χρησιμοποιούμε τον **αλγόριθμο EM** στην ειδική μορφή του για HMM, γνωστό ως **Baum–Welch**.

E-step: forward–backward, γ και ξ

Εκτός από τα forward $\alpha_t(j)$, ορίζουμε τα backward:

$$\beta_t(j) = P(O_{t+1}, \dots, O_T \mid q_t = j).$$

Αρχικοποίηση.

$$\beta_T(j) = 1.$$

Recursion.

$$\beta_t(i) = \sum_{j=1}^S a_{ij} b_j(O_{t+1}) \beta_{t+1}(j).$$

Με αυτά, υπολογίζουμε:

$$\gamma_t(i) = P(q_t = i \mid O_{0:T}) = \frac{\alpha_t(i) \beta_t(i)}{P(O_{0:T})},$$

$$\xi_t(i, j) = P(q_t = i, q_{t+1} = j \mid O_{0:T}) = \frac{\alpha_t(i) a_{ij} b_j(O_{t+1}) \beta_{t+1}(j)}{P(O_{0:T})}.$$

Ερμηνείες:

- $\gamma_t(i)$: αναμενόμενη “παρουσία” στην κατάσταση i στο χρόνο t .
- $\xi_t(i, j)$: αναμενόμενος αριθμός μεταβάσεων $i \rightarrow j$ μεταξύ t και $t + 1$.

M-step: ενημέρωση παραμέτρων

- Αρχικές πιθανότητες:

$$\pi_i^{\text{new}} = \gamma_0(i).$$

- Μεταβατικές πιθανότητες:

$$a_{ij}^{\text{new}} = \frac{\sum_{t=0}^{T-1} \xi_t(i, j)}{\sum_{t=0}^{T-1} \gamma_t(i)}.$$

- Πιθανότητες εκπομπής (διακριτές παρατηρήσεις):

$$b_j^{\text{new}}(o) = \frac{\sum_{t: O_t=o} \gamma_t(j)}{\sum_{t=0}^T \gamma_t(j)}.$$

Το EM αυξάνει μονοτονικά το log-likelihood, αλλά μπορεί να καταλήξει σε **τοπικά** optima.

2.7 Viterbi-training (hard EM) και σύγκριση

Το Viterbi-training είναι μια απλοποιημένη (hard) εκδοχή του EM:

1. **Hard E-step**: τρέχουμε Viterbi για να βρούμε μια διαδρομή $q_{0:T}^*$ μέγιστης πιθανότητας.
2. **M-step**: θεωρούμε αυτή τη διαδρομή ως “παρατηρημένη”: μετράμε
 - πόσες φορές βρισκόμαστε σε κάθε κατάσταση,
 - πόσες μεταβάσεις $i \rightarrow j$ γίνονται,
 - από κάθε κατάσταση πόσες φορές εκπέμπεται κάθε σύμβολο,

και ενημερώνουμε A, B ως σχετικές συχνότητες.

Σύγκριση:

- **Baum-Welch (forward-backward)**:

- soft counts (χρησιμοποιεί όλες τις διαδρομές, σταθμισμένες με τις πιθανότητές τους),
- στατιστικά πιο ορθό (πραγματικό EM),
- καλύτερη συμπεριφορά όταν το μοντέλο είναι ασαφές/οι διαδρομές πολλές.

- **Viterbi-training**:

- hard assignment: χρησιμοποιεί μόνο την καλύτερη διαδρομή,
- υπολογιστικά πιο απλό και γρήγορο,
- μπορεί να παγιδευτεί πιο εύκολα σε “κακές” λύσεις.

2.8 Σχέση HMM με τα μοντέλα του Μέρους 1

- Όπως οι Γκαουσιανές στο Μέρος 1, τα HMM είναι **γενετικά μοντέλα**: δίνουν ένα πλήρες $p(O_{0:T}, q_{0:T})$.
- Όπως στο 1-D μοντέλο μάθαμε παραμέτρους με ML (μέσους, διασπορές), εδώ μαθαίνουμε παραμέτρους λ με EM.
- Η λογική “**latent μεταβλητές + EM**” είναι κοινή:
 - στο 1-D μπορούμε να έχουμε άγνωστες κλάσεις,
 - στο HMM έχουμε κρυφές καταστάσεις σε κάθε χρονική στιγμή.
- Σε αντίθεση με Perceptron/LMS, τα HMM δεν δίνουν άμεσα ένα *decision boundary* σε χώρο χαρακτηριστικών, αλλά ένα *πιθανοτικό μοντέλο ακολουθιών*, από το οποίο μπορούμε να παράγουμε ταξινομητές (π.χ. μοντέλα ανά λέξη στην αναγνώριση ομιλίας).

3 Μέρος 3: Backpropagation σε Απλό Νευρωνικό Δίκτυο

Σκοπός αυτού του μέρους είναι να κατανοήσουμε σε θεωρητικό επίπεδο το **backpropagation**:

- πώς συνδέεται με τον **κανόνα αλυσίδας** στις παραγώγους,
- πώς γενικεύει την ιδέα του **gradient descent** από γραμμικά μοντέλα (LMS) σε *πολυεπίπεδα* νευρωνικά δίκτυα,
- πώς τοπικές παράγωγοι συνδυάζονται για να δώσουν $\partial L / \partial w$ για κάθε βάρος.

3.1 Νευρωνικά δίκτυα ως διακριτικά, μη γραμμικά μοντέλα

Ένα απλό feedforward νευρωνικό δίκτυο (MLP) μπορεί να θεωρηθεί ως σύνθεση:

$$\text{είσοδος} \xrightarrow{\text{γραμμικός μετασχηματισμός}} \xrightarrow{\text{μη γραμμική ενεργοποίηση}} \xrightarrow{\text{γραμμικός μετασχηματισμός}} \dots \rightarrow \hat{y}.$$

Βασικά στοιχεία:

- **Βάρη** και **bias** στα άκρα κάθε σύνδεσης.
- Μη γραμμική ενεργοποίηση (π.χ. logistic):

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

- Συνάρτηση κόστους $L(y, \hat{y})$ (π.χ. MSE ή cross-entropy).

Σε αντίθεση με Perceptron/LMS:

- Έχουμε **πολλαπλά επίπεδα** (hidden layers).
- Το μοντέλο μπορεί να προσεγγίσει *πολύπλοκα μη γραμμικά* decision boundaries (θεωρία καθολικής προσέγγισης).

Παραμένει όμως **διακριτικό** μοντέλο: δίνει άμεσα $f(x) \approx P(y | x)$ και όχι $p(x | y)$.

3.2 Υπολογιστικός γράφος και κανόνας αλυσίδας

Το δίκτυο μπορεί να ιδωθεί ως **υπολογιστικός γράφος**:

- Κάθε κόμβος υπολογίζει μια συνάρτηση των εισόδων του.
- Η έξοδος $\hat{y}$ είναι συνάρτηση όλων των βαρών.
- Το κόστος L είναι τελικά συνάρτηση των βαρών:

$$L = L(w_1, w_2, \dots).$$

Για να εκπαιδεύσουμε το δίκτυο με gradient descent, χρειαζόμαστε

$$\frac{\partial L}{\partial w_i} \quad \text{για όλα τα βάρη } w_i.$$

Ο **κανόνας αλυσίδας** μας λέει ότι η παράγωγος ως προς ένα βάρος ισούται με το γινόμενο όλων των κατάλληλων *τοπικών παραγώγων* που συνδέουν αυτό το βάρος με την έξοδο L .

Το backpropagation είναι απλώς ένας **αποδοτικός αλγόριθμος** που:

- κάνει ένα **forward pass** (υπολογίζει όλες τις ενδιάμεσες τιμές),
- και μετά ένα **backward pass** (υπολογίζει συστηματικά τις παραγώγους).

3.3 Τυπικό 2-στρώσης δίκτυο και συνάρτηση κόστους

Έστω δίκτυο με είσοδο $\mathbf{x} = (x_1, x_2, x_3, x_4)$, ένα κρυφό νευρώνα h_1 και μία έξοδο $\hat{y}$ (για απλότητα):

- **Hidden layer:**

$$s_1^{(h)} = w_1 x_1 + w_2 x_2 + w_3 x_3 + w_4 x_4 + b_1^{(h)},$$
$$h_1 = \sigma(s_1^{(h)}).$$

- **Output layer (συνοπτικά):**

$$s^{(y)} = v_1 h_1 + \dots + b^{(y)}, \quad \hat{y} = \sigma(s^{(y)}).$$

Ως συνάρτηση κόστους παίρνουμε (για ένα δείγμα):

$$L(y, \hat{y}) = (y - \hat{y})^2.$$

Στόχος: να βρούμε

$$\frac{\partial L}{\partial w_1},$$

δηλαδή πόσο αλλάζει το κόστος αν μεταβάλουμε λίγο το βάρος w_1 .

3.4 Αναλυτικός υπολογισμός $\partial L / \partial w_1$

Εφαρμόζουμε συστηματικά τον κανόνα αλυσίδας:

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial s^{(y)}} \frac{\partial s^{(y)}}{\partial h_1} \frac{\partial h_1}{\partial s_1^{(h)}} \frac{\partial s_1^{(h)}}{\partial w_1}.$$

Υπολογίζουμε κάθε παράγοντα:

1. Από $L = (y - \hat{y})^2$:

$$\frac{\partial L}{\partial \hat{y}} = 2(\hat{y} - y).$$

2. Logistic παράγωγος:

$$\frac{\partial \hat{y}}{\partial s^{(y)}} = \sigma'(s^{(y)}) = \hat{y}(1 - \hat{y}).$$

3. Από $s^{(y)} = v_1 h_1 + \dots + b^{(y)}$:

$$\frac{\partial s^{(y)}}{\partial h_1} = v_1.$$

4. Από $h_1 = \sigma(s_1^{(h)})$:

$$\frac{\partial h_1}{\partial s_1^{(h)}} = \sigma'(s_1^{(h)}) = h_1(1 - h_1).$$

5. Από $s_1^{(h)} = w_1 x_1 + \dots + b_1^{(h)}$:

$$\frac{\partial s_1^{(h)}}{\partial w_1} = x_1.$$

Συνδυάζοντας:

$$\frac{\partial L}{\partial w_1} = 2(\hat{y} - y) \hat{y}(1 - \hat{y}) v_1 h_1(1 - h_1) x_1.$$

Αυτή είναι η **τυπική μορφή** του gradient:

gradient στο βάρος = (σφάλμα στην έξοδο) · (αλυσίδα τοπικών παραγώγων) · (είσοδος στη σύνδεση).

3.5 Διατύπωση με σήματα σφάλματος δ (error signals)

Για ευκολία γράφουμε:

- Σφάλμα εξόδου:

$$\delta^{(y)} = \frac{\partial L}{\partial s^{(y)}} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial s^{(y)}} = 2(\hat{y} - y) \hat{y}(1 - \hat{y}).$$

- Σφάλμα κρυφού νευρώνα:

$$\delta_1^{(h)} = \frac{\partial L}{\partial s_1^{(h)}} = \delta^{(y)} v_1 h_1(1 - h_1).$$

Τότε:

$$\frac{\partial L}{\partial w_1} = \delta_1^{(h)} x_1.$$

Αυτό δείχνει πολύ καθαρά ότι:

- τα δ **σφάλματα** διαδίδονται προς τα πίσω στο δίκτυο,
- κάθε gradient βάρους είναι απλώς **σφάλμα στον κόμβο-στόχο επί είσοδο στη συγκεκριμένη σύνδεση**.

3.6 Συγκρίσεις με LMS, Perceptron και HMM

- Το backpropagation είναι ακριβώς η γενίκευση της ιδέας του **gradient descent** από γραμμικά μοντέλα (LMS) σε βαθιά, μη γραμμικά δίκτυα.
- Όπως στο LMS, ορίζουμε μια συνάρτηση κόστους L και ενημερώνουμε βάρη $w \leftarrow w - \eta \frac{\partial L}{\partial w}$.
- Σε αντίθεση με το Perceptron, δεν βασιζόμαστε σε μονοτονική ενημέρωση μόνο όταν υπάρχει λάθος, αλλά σε **συνεχή βελτιστοποίηση** του κόστους μέσω παραγώγων.
- Σε αντίθεση με τα γενετικά μοντέλα (Γκαουσιανές, HMM), τα MLP είναι καθαρά **διακριτικά**: δεν μοντελοποιούν $p(x | y)$, αλλά μάθουν έμμεσα $P(y | x)$.

3.7 Σημεία-κλειδιά για θεωρητικές ερωτήσεις backprop

- Backpropagation = **κανόνας αλυσίδας + υπολογιστικός γράφος**.
- Απαιτεί:

- forward pass για τις ενδιάμεσες τιμές,
- backward pass για τα δ σφάλματα.

- Για logistic:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)).$$

- Για MSE:

$$\frac{\partial L}{\partial \hat{y}} = 2(\hat{y} - y).$$

- Κάθε gradient βάρους γράφεται ως:

$$\frac{\partial L}{\partial w} = \delta_{\text{κόμβος προορισμού}} \cdot \text{είσοδος στη σύνδεση}.$$

- Η φιλοσοφία είναι ίδια με το LMS, αλλά με **πολλά επίπεδα και μη γραμμικές ενεργοποιήσεις**, άρα πολύ μεγαλύτερη εκφραστικότητα και ικανότητα προσέγγισης πολύπλοκων ορίων απόφασης.

4 Μέρος 4: Expectation–Maximization (EM)

4.1 Θεωρητικό πλαίσιο: ML με κρυφές μεταβλητές

Ο στόχος της μέγιστης πιθανοφάνειας (ML) είναι να βρούμε παραμέτρους θ που μεγιστοποιούν τη log-likelihood των **παρατηρούμενων** δεδομένων:

$$\ell(\theta) = \log p(X | \theta).$$

Όταν όμως το μοντέλο περιέχει **κρυφές (latent) μεταβλητές** Z , γράφουμε

$$p(X | \theta) = \sum_Z p(X, Z | \theta),$$

οπότε

$$\ell(\theta) = \log \sum_Z p(X, Z | \theta).$$

Η log ενός αθροίσματος είναι δύσκολο να βελτιστοποιηθεί άμεσα, σε αντίθεση με την περίπτωση **πλήρων δεδομένων**, όπου θα είχαμε

$$\log p(X, Z | \theta).$$

Βασικές έννοιες:

- **Complete data:** τι θα βλέπαμε αν γνωρίζαμε και τις κρυφές μεταβλητές Z .
- **Incomplete / observed data:** μόνο τα X (κάποια πεδία λείπουν ή είναι latent).
- **Κρυφές μεταβλητές Z :** συμπληρώνουν τα δεδομένα έτσι ώστε να απλοποιείται η ML.

Η ιδέα του EM είναι να χρησιμοποιήσει την **αναμενόμενη** log-likelihood των πλήρων δεδομένων:

$$Q(\theta, \theta^{\text{old}}) = \mathbb{E}_{Z|X, \theta^{\text{old}}} [\log p(X, Z | \theta)].$$

E-step: Υπολογίζουμε τη συνάρτηση

$$Q(\theta, \theta^{\text{old}}) = \mathbb{E}_{Z|X, \theta^{\text{old}}} [\log p(X, Z | \theta)],$$

δηλαδή την αναμενόμενη **πλήρη** log-likelihood, όπου η expectation είναι ως προς την τρέχουσα posterior των latent Z .

M-step: Επιλέγουμε

$$\theta^{\text{new}} = \arg \max_{\theta} Q(\theta, \theta^{\text{old}}),$$

σαν να είχαμε πραγματικά τα πλήρη δεδομένα.

Επαναλαμβάνουμε τα βήματα E–M μέχρι σύγκλιση. Βαθύτερο θεωρητικό αποτέλεσμα: κάθε πλήρης επανάληψη EM **δεν μειώνει** την $\log p(X | \theta)$ (μονοτονική αύξηση μέχρι κάποιο τοπικό μέγιστο).

Τυπικές εφαρμογές:

- Μείγματα κατανομών (mixture models).
- HMM (Baum–Welch).
- Προβλήματα με **ελλιπή δεδομένα** (missing values).

4.2 Το συγκεκριμένο παράδειγμα: εκθετική $\times$ ομοιόμορφη

Στην άσκηση, τα δεδομένα είναι:

$$D = \{(1, 2), (4, 5), (2, *)\},$$

όπου το $*$ δηλώνει άγνωστη τιμή για x_2 στο τρίτο δείγμα.

Υποθέτουμε ανεξαρτησία:

$$p(x_1, x_2 | \theta) = p(x_1 | \theta_1) p(x_2 | \theta_2),$$

με:

- **Εκθετική** για x_1 (με μέσο θ_1):

$$p(x_1 | \theta_1) = \begin{cases} \frac{1}{\theta_1} \exp\left(-\frac{x_1}{\theta_1}\right), & x_1 \geq 0, \\ 0, & \text{αλλιώς.} \end{cases}$$

- **Ομοιόμορφη** για x_2 :

$$p(x_2 | \theta_2) = U(0, \theta_2) = \begin{cases} \frac{1}{\theta_2}, & 0 \leq x_2 \leq \theta_2, \\ 0, & \text{αλλιώς,} \end{cases}$$

όπου $\theta_2 > 0$.

Παράμετροι:

$$\theta = (\theta_1, \theta_2).$$

Κρυφή μεταβλητή:

$$Z := x_2^{(3)},$$

η άγνωστη τιμή του x_2 στο τρίτο δείγμα.

Η άσκηση ζητά να εφαρμόσουμε ένα **ένα** βήμα EM, με αρχική εκτίμηση

$$\theta^{(0)} = (\theta_1^{(0)}, \theta_2^{(0)}) = (2, 3).$$

4.3 (α) E-step: κατασκευή του $Q(\theta, \theta^{(0)})$

Πλήρη δεδομένα. Αν γνωρίζαμε και το Z , τα πλήρη δεδομένα θα ήταν:

$$\tilde{D} = \{(1, 2), (4, 5), (2, Z)\}.$$

Λόγω ανεξαρτησίας:

$$p(\tilde{D} | \theta) = \prod_{n=1}^3 p(x_1^{(n)} | \theta_1) p(x_2^{(n)} | \theta_2).$$

Άρα

$$\log p(\tilde{D} | \theta) = \sum_{n=1}^3 \log p(x_1^{(n)} | \theta_1) + \sum_{n=1}^3 \log p(x_2^{(n)} | \theta_2).$$

Μέρος για x_1 (χωρίς κρυφές μεταβλητές). Τα x_1 είναι 1, 4, 2. Για την εκθετική:

$$\log p(x_1^{(n)} | \theta_1) = -\log \theta_1 - \frac{x_1^{(n)}}{\theta_1}.$$

Άρα:

$$\sum_{n=1}^3 \log p(x_1^{(n)} | \theta_1) = -3 \log \theta_1 - \frac{1+4+2}{\theta_1} = -3 \log \theta_1 - \frac{7}{\theta_1}.$$

Δεν εξαρτάται από το Z , άρα το μέρος αυτό είναι **ίδιο** με απλό ML.

Μέρος για x_2 (εξαρτάται από Z). Τα πλήρη x_2 θα ήταν 2, 5, Z . Για την $U(0, \theta_2)$:

$$\log p(x_2^{(n)} | \theta_2) = -\log \theta_2 + \log \mathbf{1}_{[0, \theta_2]}(x_2^{(n)}),$$

όπου $\mathbf{1}_{[0, \theta_2]}$ είναι η ενδεικτική του διαστήματος. Συνεπώς:

$$\sum_{n=1}^3 \log p(x_2^{(n)} | \theta_2) = -3 \log \theta_2 + \sum_{n=1}^3 \log \mathbf{1}_{[0, \theta_2]}(x_2^{(n)}).$$

Για να μην γίνει η log-likelihood $-\infty$, πρέπει

$$0 \leq 2, 5, Z \leq \theta_2,$$

δηλαδή

$$\theta_2 \geq \max\{2, 5, Z\}.$$

Ορισμός του $Q(\theta, \theta^{(0)})$. Με βάση τον ορισμό:

$$Q(\theta, \theta^{(0)}) = \mathbb{E}_{Z|D_{\text{obs}}, \theta^{(0)}} [\log p(\tilde{D} | \theta)].$$

Ο παράγοντας για x_1 δεν εξαρτάται από Z , άρα:

$$Q(\theta, \theta^{(0)}) = Q_1(\theta_1) + Q_2(\theta_2, \theta_2^{(0)}),$$

με

$$Q_1(\theta_1) = -3 \log \theta_1 - \frac{7}{\theta_1}.$$

Για το Q_2 :

$$Q_2(\theta_2, \theta_2^{(0)}) = \mathbb{E}_{Z|\theta_2^{(0)}} \left[-3 \log \theta_2 + \sum_{n=1}^3 \log \mathbf{1}_{[0, \theta_2]}(x_2^{(n)}) \right].$$

Η κατανομή του Z υπό $\theta^{(0)}$ είναι:

$$Z \sim U(0, \theta_2^{(0)}) = U(0, 3).$$

Αν επιλέξουμε $\theta_2 < 5$ ή $\theta_2 < \theta_2^{(0)}$, τότε υπάρχει μη μηδενική πιθανότητα το Z να βρεθεί εκτός $[0, \theta_2]$, οπότε η αναμενόμενη log-likelihood γίνεται $-\infty$. Συνεπώς, για να είναι Q_2 πεπερασμένο πρέπει:

$$\theta_2 \geq \max\{5, \theta_2^{(0)}\}.$$

Σε αυτό το επιτρεπτό πεδίο, οι ενδεικτικές γίνονται 1 σχεδόν σίγουρα ως προς Z , άρα:

$$Q_2(\theta_2, \theta_2^{(0)}) = -3 \log \theta_2 + \text{σταθερά ανεξάρτητη από } \theta_2.$$

Συνολικά (αγνοώντας σταθερές ως προς τη μεγιστοποίηση):

$$Q(\theta, \theta^{(0)}) = -3 \log \theta_1 - \frac{7}{\theta_1} - 3 \log \theta_2,$$

με περιορισμό $\theta_2 \geq \max\{5, \theta_2^{(0)}\}$.

4.4 (β) M-step: ML εκτίμηση παραμέτρων

Βελτιστοποίηση ως προς θ_1 . Με

$$Q_1(\theta_1) = -3 \log \theta_1 - \frac{7}{\theta_1},$$

έχουμε:

$$\frac{\partial Q_1}{\partial \theta_1} = -\frac{3}{\theta_1} + \frac{7}{\theta_1^2}.$$

Θέτοντας ίσο με μηδέν:

$$-\frac{3}{\theta_1} + \frac{7}{\theta_1^2} = 0 \Rightarrow -3\theta_1 + 7 = 0 \Rightarrow \boxed{\theta_1^{\text{new}} = \frac{7}{3}}.$$

Θεωρητικά:

$$\theta_1^{\text{ML}} = \text{μέση τιμή των } x_1 = \frac{1 + 4 + 2}{3} = \frac{7}{3},$$

δηλαδή ακριβώς ML για εκθετική με παράμετρο κλίμακας.

Βελτιστοποίηση ως προς θ_2 . Μέσα στο επιτρεπτό πεδίο $\theta_2 \geq \max\{5, \theta_2^{(0)}\}$, έχουμε:

$$Q_2(\theta_2) = -3 \log \theta_2 + \text{σταθερά}.$$

Επειδή $-\log \theta_2$ είναι φθίνουσα συνάρτηση, το Q_2 μεγιστοποιείται στη **μικρότερη** επιτρεπτή τιμή:

$$\boxed{\theta_2^{\text{new}} = \max\{5, \theta_2^{(0)}\}}.$$

Με $\theta_2^{(0)} = 3$:

$$\boxed{\theta_2^{(1)} = 5}.$$

Συνολική ενημέρωση:

$$\theta^{(1)} = \left(\frac{7}{3}, 5 \right).$$

Παρατηρούμε:

- Για την εκθετική, η EM απλώς **συμπίπτει** με το κλασικό ML.
- Για την ομοιόμορφη, η EM “μαθαίνει” ότι η στήριξη πρέπει να καλύπτει τη μέγιστη παρατήρηση $x_2 = 5$, άρα $\theta_2^{\text{ML}} = \max x_2^{(n)} = 5$.

4.5 (γ) Γραφική ερμηνεία και σύνδεση με άλλα μοντέλα

Πριν το EM: $\theta^{(0)} = (2, 3)$.

- $p(x_1 | \theta_1^{(0)}) = \frac{1}{2}e^{-x_1/2}$: εκθετική με μέσο 2 (πιο “απότομη”).
- $p(x_2 | \theta_2^{(0)}) = U(0, 3)$:

$$p(x_2) = \begin{cases} \frac{1}{3}, & 0 \leq x_2 \leq 3, \\ 0, & \text{αλλιώς.} \end{cases}$$

Δεν καλύπτει τη τιμή $x_2 = 5$.

Μετά το EM: $\theta^{(1)} = (7/3, 5)$.

- $p(x_1 | \theta_1^{(1)}) = \frac{3}{7}e^{-3x_1/7}$: μέσος $\frac{7}{3} \approx 2.33$, άρα η κατανομή “απλώνεται” ελαφρώς.
- $p(x_2 | \theta_2^{(1)}) = U(0, 5)$:

$$p(x_2) = \begin{cases} \frac{1}{5}, & 0 \leq x_2 \leq 5, \\ 0, & \text{αλλιώς,} \end{cases}$$

το διάστημα μεγαλώνει ώστε να περιλάβει τη μέγιστη παρατήρηση, και το ύψος της πυκνότητας μειώνεται αντίστοιχα.

Σύνδεση με άλλα μέρη.

- Όπως στο Μέρος 1 κάνουμε ML για Γκαουσιανές, εδώ κάνουμε ML με **latent** ποσότητες, άρα χρειάζεται EM.
- Όπως στα HMM (Μέρος 2) τα κρυφά states q_t οδηγούν σε forward-backward, εδώ το κρυφό Z οδηγεί σε E-step με expectation ως προς $p(Z | X, \theta^{(0)})$.
- Η φιλοσοφία “latent + EM” είναι κοινή για **μείγματα, HMM** και μοντέλα με ελλιπή δεδομένα.

5 Μέρος 5: Bayes Network – Μοντέλο Chest Clinic

5.1 Θεωρία Bayesian Networks

Ένα **Bayesian Network (BN)** είναι ένας κατευθυνόμενος ακυκλικός γράφος (DAG) που περιγράφει την joint κατανομή ενός συνόλου τυχαίων μεταβλητών $\{X_1, \dots, X_n\}$ μέσω:

$$P(X_1, \dots, X_n) = \prod_{i=1}^n P(X_i \mid \text{γονείς}(X_i)).$$

Κύρια χαρακτηριστικά:

- Κάθε ακμή $U \rightarrow V$ δηλώνει *άμεση πιθανοτική εξάρτηση*.
- Η δομή του γράφου κωδικοποιεί **συνθήκες ανεξαρτησίας** μέσω της έννοιας **d-separation**.
- Το BN επιτρέπει:
 - **forward reasoning** (από αιτίες σε συμπτώματα),
 - **diagnostic reasoning** (από συμπτώματα σε αιτίες),
 - **intercausal** επεξηγήσεις (π.χ. colliders).

Τρεις βασικοί τύποι μικρο-δομών:

- *Chain*: $X \rightarrow Y \rightarrow Z$.
- *Fork*: $X \leftarrow Y \rightarrow Z$.
- *Collider*: $X \rightarrow Y \leftarrow Z$.

Η θεωρία d-separation λέει τότε δύο κόμβοι είναι συνθηκά ανεξάρτητοι δεδομένου ενός συνόλου Z :

- Αν σε κάθε path ανάμεσά τους υπάρχει:
 - μη-collider κόμβος που ανήκει στο Z , ή
 - collider κόμβος τέτοιος ώστε ούτε αυτός ούτε απόγονός του ανήκει στο Z ,τότε οι κόμβοι είναι **d-separated** και άρα συνθηκά ανεξάρτητοι.

5.2 Το μοντέλο Chest Clinic ως παράδειγμα BN

Το Chest Clinic BN μοντελοποιεί απλοποιημένη ιατρική διάγνωση για αναπνευστικά προβλήματα. Κόμβοι:

- a : επίσκεψη στην περιοχή Ω (visit to Asia).
- s : καπνιστής (smoker).
- t : φυματίωση (tuberculosis).
- l : καρκίνος του πνεύμονα (lung cancer).
- b : βρογχίτιδα (bronchitis).
- e : “either” (τουλάχιστον ένα από t, l είναι true).
- x : αποτέλεσμα ακτινογραφίας (X-ray).

- d : δύσπνοια (dyspnoea).

Δομή:

- $a \rightarrow t$ (visit επηρεάζει την πιθανότητα φυματίωσης).
- $s \rightarrow l, s \rightarrow b$ (smoking επηρεάζει cancer & bronchitis).
- $t \rightarrow e, l \rightarrow e$ (either αν υπάρχει t, l ή και τα δύο).
- $e \rightarrow x, e \rightarrow d$ (η ύπαρξη σοβαρής νόσου επηρεάζει ακτινογραφία και δύσπνοια).
- $b \rightarrow d$ (bronchitis επηρεάζει δύσπνοια).

5.3 (α) Παραγοντοποίηση της joint κατανομής

Από τον ορισμό BN:

$$P(a, s, t, l, b, e, x, d) = P(a) P(s) P(t | a) P(l | s) P(b | s) P(e | t, l) P(x | e) P(d | e, b).$$

Αυτό είναι το **θεωρητικό πρότυπο**: κάθε κόμβος συνοδεύεται από έναν παράγοντα $P(\text{κόμβος} | \text{γονείς})$.

5.4 (β) Συνθηκές ανεξαρτησίας και d-separation

Θεωρούμε δύο δηλώσεις:

1. $l \perp\!\!\!\perp b | s$;
2. $a \perp\!\!\!\perp s | l$;

1) $l \perp\!\!\!\perp b | s$; Ο βασικός path είναι:

$$l \leftarrow s \rightarrow b,$$

δηλαδή ένας **fork** με “γιαγιά” τον s .

- Χωρίς conditioning: l και b είναι εξαρτημένα μέσω του s (το smoking επηρεάζει και τα δύο).
- Με conditioning στο s : σε fork, αν **γνωρίζουμε** την κοινή αιτία, οι απόγονοι γίνονται συνθηκά ανεξάρτητοι.

Άρα:

$$l \perp\!\!\!\perp b | s.$$

2) $a \perp\!\!\!\perp s | l$; Κοιτάμε paths από a σε s . Ένας σημαντικός είναι:

$$a \rightarrow t \rightarrow e \leftarrow l \leftarrow s.$$

Υπάρχει collider στη θέση e ($t \rightarrow e \leftarrow l$). Ο l είναι μη-collider στο path (βέλος από s και προς e).

- Χωρίς conditioning: ο collider e **μπλοκάρει** το path, άρα a και s είναι ανεξάρτητοι.
- Με conditioning στο l (μη-collider) εξακολουθούμε να έχουμε μπλοκάρισμα στον e (δεν παρατηρούμε τον collider ούτε απόγονό του), άρα το path παραμένει μπλοκαρισμένο.

Δεν υπάρχουν άλλοι ενεργοί paths που να “ανοίγουν” εξάρτηση, άρα:

$$a \perp\!\!\!\perp s | l.$$

Ερμηνεία: αν ήδη γνωρίζουμε αν ο ασθενής έχει καρκίνο (l), η πληροφορία για το αν επισκέφθηκε την περιοχή Ω (a) δεν μας δίνει παραπάνω πληροφορία για το αν είναι καπνιστής (s), και αντίστροφα.

5.5 (γ) Υπολογισμός πιθανοτήτων με product-sum

Η factorization

$$P(a, s, t, l, b, e, x, d) = P(a) P(s) P(t | a) P(l | s) P(b | s) P(e | t, l) P(x | e) P(d | e, b)$$

μας επιτρέπει να υπολογίσουμε οποιαδήποτε περιθωριακή ή συνθήκη πιθανότητα με τον γενικό κανόνα **sum-product**:

$$P(\text{μεταβλητές ενδιαφέροντος}) = \sum_{\text{λοιπές μεταβλητές παράγοντες } \phi} \prod \phi(\text{μεταβλητές } \phi).$$

Ενδεικτικά:

•

$$P(x = T) = \sum_{a,s,t,l,b,e,d} P(a) P(s) P(t | a) P(l | s) P(b | s) P(e | t, l) P(x = T | e) P(d | e, b).$$

•

$$P(x = T | t = F) = \frac{P(x = T, t = F)}{P(t = F)},$$

όπου

$$P(x = T, t = F) = \sum_{a,s,l,b,e,d} P(a, s, t = F, l, b, e, x = T, d).$$

•

$$P(t = T | x = F) = \frac{P(t = T, x = F)}{P(x = F)}.$$

•

$$P(t = T | x = F, d = T) = \frac{P(t = T, x = F, d = T)}{P(x = F, d = T)}.$$

Στην πράξη, οι αριθμητικοί υπολογισμοί γίνονται με **message passing** σε κατάλληλη δομή (junction tree), όχι με «ωμό» άθροισμα σε όλες τις 2^8 περιπτώσεις.

5.6 Moralization, triangulation και αποδοτικό inference

Για μεγάλα δίκτυα, το απλό sum-product είναι εκθετικό. Η κλασική θεωρητική διαδικασία για αποδοτικό exact inference είναι:

1. Moralization:

- Ενώνουμε (παντρεύουμε) όλους τους γονείς κάθε κόμβου (π.χ. $t-l$ για τον e).
- Κάνουμε τον γράφο μη κατευθυνόμενο.

2. Triangulation:

- Προσθέτουμε ακμές ώστε να μην υπάρχουν κύκλοι ≥ 4 χωρίς χορδή.
- Προκύπτουν **cliques**, στις οποίες ορίζουμε potentials.

3. Junction tree & message passing:

- Χτίζουμε ένα δέντρο από cliques.
- Πολλαπλασιάζουμε τους παράγοντες $P(\cdot | \cdot)$ στις κατάλληλες cliques.
- Εφαρμόζουμε sum-product (messages) για να πάρουμε όλες τις ζητούμενες περιθωριακές/posterior πιθανότητες.

Η λογική είναι ανάλογη με forward-backward στα HMM: **τοπικά προϊόντα + άθροισμα πάνω σε κρυφές μεταβλητές**.

5.7 Σύνδεση με τα υπόλοιπα μέρη

- Όπως στον Bayes ταξινομητή (Μέρος 1) χρειαζόμαστε $P(C \mid x)$, εδώ χρειαζόμαστε γενικές $P(\text{μέρος του γράφου} \mid \text{evidence})$.
- Όπως στο EM (Μέρος 4) γράφουμε joint κατανομές ως γινόμενο απλών παραγόντων και αθροίζουμε πάνω σε latent μεταβλητές, στα BN κάνουμε sum-product πάνω σε παράγοντες $P(X_i \mid \text{γονείς})$.
- Η ιδέα **τοπικών πιθανοτήτων + δομή γραφήματος** είναι κοινή σε HMM, BN, factor graphs και γενικά σε γραφικά μοντέλα.

6 Μέρος 6: Απλό RNN που Μετράει Μονάδες

6.1 RNN ως δυναμικό διακριτικό μοντέλο

Ένα Recurrent Neural Network (RNN) είναι ένα νευρωνικό δίκτυο που επεξεργάζεται ακολουθίες εισόδων

$$x_1, x_2, \dots, x_T$$

και διατηρεί μια εσωτερική κατάσταση s_k στο χρόνο k .

Γενική ιδέα:

$$s_k = f(s_{k-1}, x_k; \theta), \quad y_k = g(s_k; \theta),$$

όπου f, g είναι συνήθως μη γραμμικές συναρτήσεις (π.χ. tanh, ReLU κ.λπ.).

- Το RNN είναι **διακριτικό** μοντέλο: μαθαίνει απευθείας y (ή $P(y \mid x_{1:T})$), όχι $p(x_{1:T})$.
- Σε αντίθεση με τα HMM (γενετικά πιθανοτικά μοντέλα), τα RNN έχουν *ντετερμινιστική* κατάσταση με βάρη.
- Σε αντίθεση με MLP (feedforward), τα RNN είναι **δυναμικά**: η ίδια συνάρτηση f εφαρμόζεται επαναληπτικά στο χρόνο.

6.2 Ειδικό μοντέλο: γραμμικό RNN που μετράει μονάδες

Στην άσκηση, έχουμε μια ιδιαίτερα απλή μορφή RNN:

$$s_k = w_{\text{rec}} s_{k-1} + w_x x_k, \quad s_0 = 0,$$

όπου:

- $x_k \in \{0, 1\}$ – δυαδική είσοδος,
- s_k – scalar hidden state,
- w_{rec} – recurrent βάρη,
- w_x – input βάρη.

Η έξοδος ορίζεται ως:

$$y = s_T,$$

δηλαδή μόνο στο τελευταίο timestep.

Ο στόχος είναι:

$$t = \sum_{k=1}^T x_k,$$

δηλαδή το **πλήθος μονάδων** στην ακολουθία.

Κόστος:

$$L(y, t) = (t - y)^2 = (t - s_T)^2.$$

Θεωρητικά, αν πάρουμε $w_{\text{rec}} = 1, w_x = 1$, τότε

$$s_k = s_{k-1} + x_k \Rightarrow s_k = \sum_{i=1}^k x_i,$$

άρα $y = s_T = t$: το RNN γίνεται ιδανικός *accumulator* μονάδων.

6.3 (α) Backpropagation Through Time (BPTT) και gradients

Θέλουμε τις παραγώγους:

$$\frac{\partial L}{\partial w_{\text{rec}}}, \quad \frac{\partial L}{\partial w_x},$$

για να εφαρμόσουμε gradient descent:

$$w_{\text{rec}} \leftarrow w_{\text{rec}} - \eta \frac{\partial L}{\partial w_{\text{rec}}}, \quad w_x \leftarrow w_x - \eta \frac{\partial L}{\partial w_x}.$$

Ο αλγόριθμος είναι **Backpropagation Through Time** (BPTT): ξετυλίγουμε το RNN στον χρόνο και εφαρμόζουμε backprop όπως σε πολύ βαθύ δίκτυο με κοινά βάρη.

1. Παράγωγος κόστους ως προς s_T . Με $L = (t - s_T)^2$:

$$\delta_T := \frac{\partial L}{\partial s_T} = 2(s_T - t).$$

2. Παράγωγος προς προηγούμενες καταστάσεις. Για $k < T$, το L εξαρτάται από s_k μέσω του s_{k+1} :

$$s_{k+1} = w_{\text{rec}} s_k + w_x x_{k+1}.$$

Άρα:

$$\delta_k := \frac{\partial L}{\partial s_k} = \frac{\partial L}{\partial s_{k+1}} \frac{\partial s_{k+1}}{\partial s_k} = \delta_{k+1} w_{\text{rec}}.$$

Επαναληπτικά:

$$\delta_k = \delta_T w_{\text{rec}}^{T-k}.$$

3. Gradient ως προς w_{rec} . Σε κάθε timestep k , η εξάρτηση από w_{rec} είναι

$$s_k = w_{\text{rec}} s_{k-1} + w_x x_k \Rightarrow \frac{\partial s_k}{\partial w_{\text{rec}}} = s_{k-1}.$$

Συνολικά:

$$\frac{\partial L}{\partial w_{\text{rec}}} = \sum_{k=1}^T \delta_k s_{k-1}.$$

4. Gradient ως προς w_x . Παρομοίως:

$$\frac{\partial s_k}{\partial w_x} = x_k,$$

άρα:

$$\frac{\partial L}{\partial w_x} = \sum_{k=1}^T \delta_k x_k.$$

5. Κανόνες ανανέωσης. Με gradient descent:

$$\begin{aligned} w_{\text{rec}} &\leftarrow w_{\text{rec}} - \eta \sum_{k=1}^T \delta_k s_{k-1}, \\ w_x &\leftarrow w_x - \eta \sum_{k=1}^T \delta_k x_k, \end{aligned}$$

όπου

$$\delta_T = 2(s_T - t), \quad \delta_k = w_{\text{rec}} \delta_{k+1}, \quad k = T-1, \dots, 1.$$

Αυτό είναι ένα πλήρες παράδειγμα BPTT για ένα γραμμικό RNN.

6.4 (β) Παράδειγμα ενός update με αρχικά βάρη

Για την ακολουθία

$$[x_1, \dots, x_6] = [1, 0, 1, 0, 1, 0], \quad T = 6, \quad t = 3,$$

και αρχικά βάρη

$$w_{\text{rec}}^{(0)} = 1, \quad w_x^{(0)} = 0,$$

ισχύει:

- **Forward:**

$$s_k^{(0)} = 1 \cdot s_{k-1}^{(0)} + 0 \cdot x_k = 0 \Rightarrow s_k^{(0)} = 0, \quad \forall k.$$

$$\text{Άρα } y^{(0)} = 0, \quad L^{(0)} = (3 - 0)^2 = 9.$$

- **Backward:**

$$\delta_6^{(0)} = 2(s_6^{(0)} - t) = -6, \quad \delta_k^{(0)} = w_{\text{rec}}^{(0)} \delta_{k+1}^{(0)} = -6, \quad \forall k.$$

- **Gradients:**

$$\left. \frac{\partial L}{\partial w_x} \right|_{(0)} = \sum_{k=1}^6 \delta_k^{(0)} x_k = -6 \cdot (1 + 0 + 1 + 0 + 1 + 0) = -18,$$

$$\left. \frac{\partial L}{\partial w_{\text{rec}}} \right|_{(0)} = \sum_{k=1}^6 \delta_k^{(0)} s_{k-1}^{(0)} = 0,$$

$$\text{επειδή όλα τα } s_{k-1}^{(0)} = 0.$$

- **Update:**

$$w_x^{(1)} = 0 - \eta(-18) = 18\eta, \quad w_{\text{rec}}^{(1)} = 1.$$

Έτσι, το πρώτο update “ανοίγει” τον δρόμο ώστε το δίκτυο να αρχίσει να μετράει τις μονάδες μέσω του w_x .

Τα επόμενα updates υπολογίζονται με την ίδια διαδικασία, τώρα όμως τα s_k είναι μη μηδενικά και το gradient ως προς w_{rec} γίνεται επίσης μη μηδενικό.

6.5 (γ) Καλύτερη αρχικοποίηση και ιδανική λύση

Ιδανικά βάρη. Για τέλειο counting:

$$w_{\text{rec}}^* = 1, \quad w_x^* = 1.$$

Τότε:

$$s_k^* = \sum_{i=1}^k x_i, \quad y^* = s_T^* = t,$$

οπότε $L = 0$ και όλα τα gradients γίνονται μηδενικά.

Καλή αρχικοποίηση στην πράξη. Συνήθως δεν γνωρίζουμε τη βέλτιστη λύση, αλλά:

$$w_{\text{rec}}^{(0)} \approx 1, \quad w_x^{(0)} \approx 1$$

είναι λογική επιλογή για *accumulator* τύπου RNN:

- $|w_{\text{rec}}| < 1$ οδηγεί σε **vanishing** μνήμη και gradients, ειδικά για μεγάλα T .
- $|w_{\text{rec}}| > 1$ οδηγεί σε **exploding** καταστάσεις και gradients.
- $w_{\text{rec}} \approx 1$ διατηρεί τις πληροφορίες για αρκετά βήματα, χωρίς να εκρήγνυνται οι τιμές.

6.6 (δ) Vanishing / exploding gradients και συγκρίσεις

Η σχέση

$$\delta_k = w_{\text{rec}}^{T-k} \delta_T$$

δείχνει καθαρά:

- Αν $|w_{\text{rec}}| < 1$, τότε $|w_{\text{rec}}^{T-k}|$ μικραίνει εκθετικά με το μήκος της ακολουθίας: **vanishing gradients**.
- Αν $|w_{\text{rec}}| > 1$, τότε μεγαλώνει εκθετικά: **exploding gradients**.

Γι' αυτό τα πιο εξελιγμένα RNN (LSTM, GRU) χρησιμοποιούν gates, skip connections και άλλες τεχνικές για να αντιμετωπίσουν αυτά τα φαινόμενα.

Συγκριτικά:

- Σε σχέση με **LMS / Perceptron** (Μέρος 1), εδώ η δομή είναι δυναμική στον χρόνο, αλλά η βασική ιδέα gradient descent είναι η ίδια.
- Σε σχέση με **HMM** (Μέρος 2), τα RNN δεν έχουν explicit κατανομές μετάβασης/εκπομπής, αλλά μια ντετερμινιστική δυναμική $s_k = f(s_{k-1}, x_k)$, την οποία μαθαίνουν από τα δεδομένα.
- Σε σχέση με **MLP και backprop** (Μέρος 3), το BPTT είναι απλώς backprop σε ένα δίκτυο που έχει επαναληπθεί T φορές με κοινά weights.

Έτσι, το απλό RNN της άσκησης είναι ένα ιδανικό θεωρητικό παράδειγμα για να κατανοήσεις:

- πώς η πληροφορία ρέει στον χρόνο,
- πώς τα gradients πολλαπλασιάζονται ξανά και ξανά με το w_{rec} ,
- πώς συνδέεται η δυναμική του RNN με τα φαινόμενα vanishing/exploding.